

TDS200 · Kryssplattform

Session 6

Comments & Expo-Camera

Comments · Likes · Array Operations

Cross-Platform Mobile Development · Lecturer: Yuan Lin

MID-TERM EVALUATION

Mid-Term Evaluation

- After talking to Per, I will make a few adjustments from now on:
 - Set up the **microphone for recording** with support from IT. There will be **no live streaming**.
 - Repeat a brief **course overview at the beginning of each session** and clearly show where the session fits into the overall course structure.
 - Place more emphasis on the **exam requirements and expectations**.
 - Start each session by highlighting **what is new**.
 - Include a short **2–5 minute warm-up coding activity** at the beginning of each session. (Will try, but can not guarantee)
 - Make **newly added code blocks and features more visible** during teaching.
 - Respond to **student messages** in-time

Exam

- 3 weeks home exam – individual or in group up to 3 candidates
- Both technical and theoretical competence will be evaluated.
- A group is expected to accomplish more for the same grade as compared to individual.

symbol	betegnelse	generell, ikke fagspesifikk beskrivelse av vurderingskriterier	Kommentarer om nivået
A	fremragende	Fremragende prestasjon som klart utmerker seg. Kandidaten viser svært god vurderingsevne og stor grad av selvstendighet.	Høy grad av måloppnåelse
B	meget god	Meget god prestasjon. Kandidaten viser meget god vurderingsevne og selvstendighet.	
C	god	Jevnt god prestasjon som er tilfredsstillende på de fleste områder. Kandidaten viser god vurderingsevne og selvstendighet på de viktigste områdene.	Middels grad av måloppnåelse
D	nokså god	En akseptabel prestasjon med noen vesentlige mangler. Kandidaten viser en viss grad av vurderingsevne og selvstendighet.	
E	tilstrekkelig	Prestasjonen tilfredsstiller minimumskravene, men heller ikke mer. Kandidaten viser liten vurderingsevne og selvstendighet.	Lav grad av måloppnåelse
F	ikke bestått	Prestasjon som ikke tilfredsstiller de faglige minimumskravene. Kandidaten viser både manglende vurderingsevne og selvstendighet.	



12 sessions × 2 hours

Lecture + coding, with a weekly assignment between sessions.



Assessment

3-week home exam



Prereq

JS/React from DS3103 — we extend it to mobile.

Exam

- **Your submission follows the structure:**

TDS200_h26_[candidate-number].zip

Your candidate number must appear on the submission

/app — runnable product

50%

- Must run in web and one of the emulators
- Source code + short readme included

Portfolio.pdf

30%

- Design, tech-stack choice, implementation, test & validation
- A dedicated reflection chapter
- AI usage documented together with the reflection

Theory.pdf

10%

- Report answering the theory chapter in the exam paper

Demo.mp4

10%

- Author navigates and explains the app (up to 5 min)

- **Your Portfolio.pdf should contain following elements:**
 - Design, tech stack choice, implementation overview, test % validation
ca. 15%
 - A dedicated reflection chapter
ca. 10%

The reflections should demonstrate critical consideration of the student's own problem-solving process and may address questions like:

- What was the main problem or challenge in the activity?
- How was the problem solved? Which technical or methodological decisions were made, and why?
- What worked well and what could have been improved?
- AI usage documentation
ca. 5%

What you'll learn today

- User interaction features that make the app more interactive and responsible
 - Add/delete/edit/display comments
 - Like/unlike
- Array field operations & API Integration
- The native-plugin ecosystem (Expo modules, Capacitor, Cordova)
- Capture photos with expo-camera



Plugins = bridges

Write JS; the plugin handles Swift/Kotlin under the hood.



Project step

Attach a photo (camera or gallery) to a new post.

The 12 sessions

1 Introduction & Foundations

2 Components & Navigation (Expo Router)

3 Components, State & Local Storage

4 Forms, Modals & Creating Data

5 Firebase Firestore CRUD

6 **Firestore CRUD & Camera**

7 Firebase Storage, Camera & Image Picker

8 Firebase Authentication & Context

9 Google sign-in, Firebase Rules & Guards

10 NativeWind & Geolocation

11 Google Maps & Queries

12 Exam & Wrap-Up

What is new in the codebase?

- We have added a new `commentApi.ts` under the `api` folder.
- We have created two new components: `CommentSection.tsx` and `CommentModal.tsx` for displaying and adding comments.
- We have integrated `CommentSection` and `CommentModal` to `postDetails/[id].tsx` to enable comments functions.
- We have updated `Post.tsx` component to show number of comments.
- We have created a new `ImageSelector.tsx` under `component` folder to capture camera.
- We have integrated `ImageSelector` to `PostForm.tsx` so that when you create a new Post, you can select an image from capturing the camera.
- We have made corresponding updates to `postDetails/[id].tsx` and `index.tsx` to show images.

How a 2-hour session runs

0:00-0:25	Recap & objectives Where we are in the project; today's goals
0:25-0:45	Array & API Integration Like/Unlike, Comment,, arrayUnion & arrayRemove
0:45-1:00	Break —
1:00-1:40	Expo-Camera Capture photos, Store as binary in Firestore
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

toggleLike

- No document read required.
- **Firestore performs the array update atomically on the server.**
- You do not need manually read-modify-write.

```
export async function toggleLike(  
  id: string,  
  authorId: string,  
  currentlyLiked: boolean  
): Promise<void> {  
  await updateDoc(doc(db, "posts", id), {  
    likes: currentlyLiked  
      ? arrayRemove(authorId)  
      : arrayUnion(authorId),  
  });  
}
```



```
export async function toggleLike(  
  id: string,  
  authorId: string,  
  currentlyLiked: boolean  
): Promise<void> {  
  const postRef = doc(db, "posts", id);  
  const snap = await getDoc(postRef);  
  const likes = snap.data()?.likes??[];  
  
  await updateDoc(postRef, {  
    likes:[...likes, authorId],  
  });  
}
```

handleToggleLike

Optimistic update:

- Flip likes array in local state immediately,
- Then persist with arrayUnion / arrayRemove in Firestore.
- Two actions in parallel.
- We do not use toggleLike(post.id, newLikes) to avoid racing condition.

```
// LIKE — optimistic update: flip likes array in local state immediately,
// then persist with arrayUnion / arrayRemove in Firestore.
async function handleToggleLike() {
  if (!post) return;
  const currentLikes = post.likes ?? [];
  const name = authorName || "Anonymous";
  const alreadyLiked = currentLikes.includes(name);
  const newLikes = alreadyLiked
    ? currentLikes.filter((n) => n !== name)
    : [...currentLikes, name];
  setPost((prev) => (prev ? { ...prev, likes: newLikes } : prev));
  await toggleLike(post.id, name, alreadyLiked);
}
```

```
// LIKE / UNLIKE — adds or removes the user's name from the post's likes array
export async function toggleLike(
  id: string,
  authorId: string,
  currentlyLiked: boolean
): Promise<void> {
  await updateDoc(doc(db, POSTS_COLLECTION, id), {
    likes: currentlyLiked ? arrayRemove(authorId) : arrayUnion(authorId),
  });
}
```

Optimistic UI – why it matters

Pessimistic

Wait for Firestore before updating the like section (number of likes, who liked etc). The UI Feels slow.

Optimistic

Update the like section immediately by updating post (state variable) and causes a re-render. The. Sync with Firebase afterwards. Feels instant.

```
async function handleToggleLike() {  
  // capture BEFORE state change  
  const alreadyLiked = currentLikes.includes(name);  
  // 1. update UI immediately  
  const newLikes = alreadyLiked  
    ? currentLikes.filter((n) => n !== name)  
    : [... currentLikes, name]  
  
  setPost((prev) => (prev ? { ...prev, likes: newLikes  
} : prev));  
  
  // 2. sync Firestore in background  
  await toggleLike(post.id, name, alreadyLiked);  
}
```

arrayUnion & arrayRemove

Call	What it does
<code>arrayUnion(value)</code>	Adds value — only if not already present
<code>arrayRemove(value)</code>	Removes value — safe if not present

Both are atomic server-side operations — help with concurrency, especially when multiple users may update the same Firestore document at nearly the same time.

IDEMPOTENT

```
// safe under concurrent writes
await updateDoc(ref, {
  likes: arrayUnion("alice"),
});
await updateDoc(ref, {
  likes: arrayUnion("alice"),
}); // still one "alice"
```

Why not read → modify → write?

UNSAFE ✕

```
const snap = await getDoc(ref);
const likes = snap.data().likes; //likes:["peter"]
await updateDoc(ref, {
  likes: [...likes, "alice"], //first user
});
```

```
const snap = await getDoc(ref);
const likes = snap.data().likes; //likes:["peter"]
await updateDoc(ref, {
  likes: [...likes, "bob"], //second user
});
```

Race condition: if two users do this at the same time, one write overwrites the other. Depending on which write arrives last, the final result could be either ["peter", "alice"], or ["peter", "bob"]

SAFE ✓

```
await updateDoc(ref, {
  likes: arrayUnion("alice"),
});
```

```
await updateDoc(ref, {
  likes: arrayUnion("bob"),
});
```

Firestore does not replace the likes array with the new array (the new array might be either ["peter", "alice"], or ["peter", "bob"]), but atomically add the new value to the existing array.

Comments design principle

- We define two interfaces:
 - **CommentData** represents data stored inside Firestore, and it contains authorId, and comment text.
 - **CommentObject** represents a fetched CommentData (authorId, comment text) together with Firestore ID. It is used locally. When you update a post object, you only insert the Firestore ID for CommentData to its comments array.
- Posts store only the IDs of CommentData.

```
posts/  
  {postId}  
    comments: ["abc123", "def456"]  
  
comments/  
  abc123 { authorId: "alice",  
           comment: "Great post!" }  
  def456 { authorId: "bob",  
           comment: "Thanks!" }
```

```
// A single comment stored in the "comments" Firestore collection  
export interface CommentData {  
  authorId: string;  
  comment: string;  
}  
  
// What we get back when we fetch a comment by id.  
export interface CommentObject {  
  id: string;  
  comment: CommentData;  
}
```

addComment

- **Two writes.**
- One creates the comment document and update comments collection in Firebase. Firebase will return an auto-generated id by **ref.id**. This ref.id can be used to fetch Comment Object.
- One updates the linked post document:
 - First, it fetches post from Firebase, and adds ref.id to the post comments array using **arrayUnion** (an array contains all Comment Object ids).
 - Then it updates Firebase with the updated post using **updateDoc**.

```
export async function addComment(  
  postId: string, comment: CommentData  
) : Promise<string | null> {  
  // 1. write the comment document  
  const ref = await addDoc(  
    collection(db, "comments"), comment);  
  // 2. link its id to the post  
  await updateDoc(doc(db, "posts", postId), {  
    comments: arrayUnion(ref.id),  
  });  
  return ref.id;  
}
```

deleteComment

- **Two operations.**
- It first remove the comment from the Post using `arrayRemove`. Note that the `commentId` is the Comment Object id, generated automatically by Firebase when you create a comment.
- then delete the comment document from Firebase using `deleteDoc`, index by `commentId`.
- Order matters: unlink first, then delete.

```
export async function deleteComment(  
  commentId: string, postId: string  
): Promise<void> {  
  // 1. remove the id from the post  
  await updateDoc(doc(db, "posts", postId), {  
    comments: arrayRemove(commentId),  
  });  
  // 2. delete the comment document  
  await deleteDoc(doc(db, "comments", commentId));  
}
```

postDetails/[id].tsx

- addComment, updateComment, deleteComment are functions defined in commentApi.ts, which directly communicates with Firestore.
- It also updates local comments, post, and posts. Then the UI gets updated immediately without fetching remotely from Firestore each time.

```
// ADD COMMENT – writes to Firestore, then updates local state immediately.
async function handleAddComment(text: string) {
  if (!post) return;
  const commentData: CommentData = {
    authorId: displayName,
    comment: text,
  };
  const newId = await addComment(post.id, commentData);
  if (newId) {
    setComments((prev) => [{ id: newId, comment: commentData }, ...prev]);
    setPost((prev) =>
      prev ? { ...prev, comments: [...(prev.comments ?? []), newId] } : prev
    );
  }
}

// EDIT COMMENT – only shown when authorId matches the current user.
async function handleEditComment(commentId: string, text: string) {
  await updateComment(commentId, { comment: text });
  setComments((prev) =>
    prev.map((c) =>
      c.id === commentId ? { ...c, comment: { ...c.comment, comment: text } } : c
    )
  );
}

// DELETE COMMENT – only shown when authorId matches the current user.
async function handleDeleteComment(commentId: string) {
  if (!post) return;
  await deleteComment(commentId, post.id);
  setComments((prev) => prev.filter((c) => c.id !== commentId));
  setPost((prev) =>
    prev
      ? { ...prev, comments: (prev.comments ?? []).filter((cid) => cid !== commentId) }
      : prev
  );
}
```

Comments Collection

> comments > vw6xQsrHTmgk...		
(default)	comments	vw6xQsrHTmgkSZjHlyxP
+ Start collection	+ Add document	+ Start collection
comments >	vw6xQsrHTmgkSZjHlyxP >	+ Add field
posts		authorId: "Anonymous" comment: "First comment"

Posts Collection



>

posts

>

pQVKuPMIHpyl...

 (default)	 posts 	 pQVKuPMIHpylBAUg3cpQ
+ Start collection comments posts >	+ Add document GJ1BXIZjvAE6mr18wpdm pQVKuPMIHpylBAUg3cpQ >	+ Start collection + Add field author: "Anonymous" ▼ comments 0 "vw6xQsrHTmgkSZjHlyxP" createdAt: 18 June 2026 at 17:07:25 UTC+2 description: "Skyyy" hashtags: "#sky #city" ▼ likes 0 "Anonymous" 1 "Tester123" title: "Sky"

RECAP

Summary

Feature	Firestore call	Pattern
Like / Unlike	<code>arrayUnion / arrayRemove on posts.likes</code>	Optimistic UI
Add comment	<code>addDoc + arrayUnion on posts.comments</code>	Two writes
Delete comment	<code>arrayRemove + deleteDoc</code>	Two writes

ARCHITECTURE

The api/ folder

- **Why do we need a separate API folder?**
 - We keep all Firestore operations inside an API folder so that screens focus only on the user interface and user interactions.
- **Now each file has a single responsibility.**
 - Screen: UI, user interaction, state management
 - API: Firestore/database operations
 - Firebase Config: Database connection and initialization

FILES

```
api/  
  postApi.ts      // CRUD + toggleLike  
  commentApi.ts   // add, read, delete
```

Security Rules

- The code controls what the UI shows. It only shows Edit/Delete buttons if the comments authorId matches the current logged in user.
- It does not enforce security. A malicious user can still bypass the UI and directly call your Firestore update/delete function.
- We need to apply Firestore Security Rules, that actually protects the data.
- For example, to prevent comments to be edited or deleted by other users, your Firestore rules should compare request.auth.uid against resource.data.authorId, which is a real server-side authorization.
- We will cover this in later sessions.

```
{loading ? (
  <ActivityIndicator size="small" style={styles.spinner} />
) : comments.length === 0 ? (
  <Text style={styles.noComments}>No comments yet.</Text>
) : (
  comments.map((item) => (
    <View key={item.id} style={styles.item}>
      <View style={styles.itemBody}>
        <Text style={styles.author}>{item.comment.authorId}</Text>
        <Text style={styles.text}>{item.comment.comment}</Text>
      </View>
      {item.comment.authorId === currentAuthor && (
        <View style={styles.itemActions}>
          <TouchableOpacity onPress={() => openEditModal(item)}>
            <Icons name="pencil" size={16} color="#007AFF" />
          </TouchableOpacity>
          <TouchableOpacity
            style={styles.deleteButton}
            onPress={() => onDeleteComment(item.id)}
          >
            <Text style={styles.deleteButtonText}>x</Text>
          </TouchableOpacity>
        </View>
      )}
    </View>
  )}
</View>
))
)}
```

Three layers of security

- Client-side authorization check.
 - Controls what the UI shows. Example: show Edit/Delete button only when `comment.authorId===currentAuthor`.
 - Improves user experience, but does not protect the database.
- .env file
 - Keeps configuration values out of your source code and Git repo. Useful for values such as `EXPO_PUBLIC_FIREBASE_API_KEY`, but in a React Native/Expo app, these values are bundled into the app and are not truly secret.
 - .env is mainly for configuration management, not database security.
- Firestore security rules.
 - Rules are checked by Firebase on the server. Example: `request.auth.uid === resource.data.authorId`. Even if a malicious user bypasses the UI or calls Firestore directly, the security rule can reject the request.
 - This is the layer that actually protects your Firestore data.

Native Plugin -expo-camera

- A native plugin connects JavaScript code to native (iOS/Android) APIs.
- Expo-camera is a native plugin:
 - It gives your React Native app access to the device camera.
 - Your JavaScript/TypeScript code can use components such as CameraView.
 - The plugin handles the native camera APIs in a transparent way.



Plugins = bridges

Write JS; the plugin handles Swift/Kotlin under the hood.



Project step

Attach a photo (camera or gallery) to a new post.

The native-plugin ecosystem



Expo / RN modules

JavaScript + React Native rendering to real native UI. expo-camera, expo-image-picker, expo-location... This course will focus on Expo/RN modules.

Native UI for UI rendering (UIKit/Swift on iOS, Views/Kotlin on Android), not a WebView



Capacitor

Web tech (HTML/CSS/JS) with your framework of choice (React/Vue/Angular) in a native shell

Plugins like @capacitor/camera.

App runs in in a WebView.



Cordova (legacy)

The original hybrid bridge.

Largely replaced by Capacitor today.

Expo-camera & Expo-image-picker

- Expo-camera is an Expo module that exposes camera functionality to React Native apps.

It provides following key features:

- CameraView component for preview
- APIs such as `takePictureAsync()` for taking photos
- Permission helpers for requesting camera access
- A single JavaScript API across iOS/Android.
- **Important: expo-camera should be tested on a real device. It does not work on an iOS simulator**
- Expo-image-picker is a high-level API to pick images from the photo gallery.
 - It can open the system image picker UI and is easier to test during development.
- Web camera and Image Gallery in simulator/emulator will be checked in the exam.

SETUP

Install expo-camera & request permission

terminal

```
npx expo install expo-camera \
  expo-image-picker
```

```
const isSimulator =
  (Platform.OS === "ios" && !Device.isDevice) ||
  (Platform.OS === "android" && !Device.isDevice);
```

- **Detect running environment**

- Device.isDevice is used to detect whether the app is running on a physical phone or a simulator/emulator.
- expo-camera is not supported for simulator, so the app disables camera capture and shows a fallback message when running in a simulator.

Camera permission

- Camera permissions in Expo requires two layers:
 - **Native permissions declaration (build time).**
 - **Runtime permission (when the feature is used).**
- With Expo Go, the native camera configuration is already included. But the runtime permission is still required.
- Before using the camera, always check/request permission:
 - `const [permission, requestPermission] = useCameraPermissions();`
- If permission is not granted, request it before showing the camera.
- Your app should handle:
 - Permission granted
 - Permission denied
 - Permission not yet requested

Capture picture from camera

- expo-camera opens the camera.
- Calling takePictureAsync() returns a uri you can render with <Image>.
- CameraView renders the camera preview.

camera permission hook

```
import { useCameraPermissions }
  from 'expo-camera';

const [permission, requestPermission] =
  useCameraPermissions();
if (! permission?.granted) requestPermission();
```

capture a photo

```
import { CameraView } from 'expo-camera';
import { useRef } from 'react';

const ref = useRef<CameraView>(null);

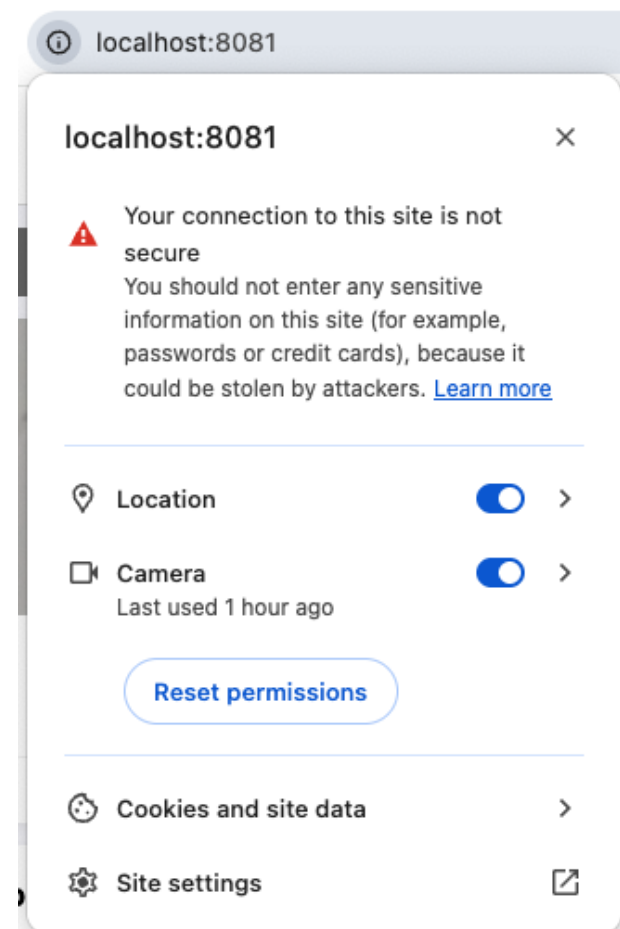
async function snap() {
  const photo = await ref.current?.takePictureAsync();
  setImageUri(photo?.uri);
}

<CameraView ref={ref} style={{ flex: 1 }} />
```

Capture picture from camera

- AT first, permission may still be loading, so you temporarily render an empty camera container.
- Then check whether permission has not been granted. If it is not granted, you press the button and request permission. Expo asks iOS/Android for camera access.
- Once permission is granted, you just show the CameraView.
- The web browser may remember your permission decision. Resetting the camera permission for the site in the browser settings will ask the user to re-grant permission.

```
if (!permission && !isSimulator) {  
  return <View style={styles.cameraContainer} />;  
}  
  
if (!isSimulator && permission && !permission.granted) {  
  return (  
    <View style={styles.cameraContainer}>  
      <Text style={styles.permissionText}>  
        We need your permission to show the camera  
      </Text>  
  
      <Button onPress={requestPermission} title="Grant permission" />  
  
      <TouchableOpacity style={styles.closeBtn} onPress={closeCamera}>  
        <Text style={styles.closeBtnText}>Close</Text>  
      </TouchableOpacity>  
    </View>  
  );  
}
```



SESSION 6 • CAMERA

Capture an image

- In ImageSelector, you add the photo.uri to images array. (it supports multiple images)
- when you submit the PostForm, handleNewPost(posts, images) will be called from index.tsx.
- It calls createPost() in postApi, and the images are inserted into the post and sent to Firestore.

```
// CREATE
export async function createPost(data: NewPost): Promise<void> {
  await addDoc(collection(db, POSTS_COLLECTION), {
    ...data,
    likes: [],
    comments: [],
    imageUrls: data.imageUrls ?? [],
    createdAt: serverTimestamp(),
  });
}
```

```
async function captureImage() {
  const photo = await cameraRef.current?.takePictureAsync({
    quality: 0.85,
  });

  if (photo?.uri) {
    onChange([...images, photo.uri]);
    closeCamera();
  }
}
```

```
/* Pictures – only when creating a new post, not when editing one */
{!isEditMode && [
  <Text style={styles.label}>Images</Text>
  <ImageSelector images={images} onChange={setImages} />
]}
</>
```

```
// CREATE POST – localImages are the URIs the camera and gallery gave us.
// We save them straight into Firestore, so they only show on this device.
// Session 7 uploads the files to Firebase Storage first and saves real URLs.
async function handleNewPost(post: PostData, localImages: string[]) {
  try {
    await createPost({
      title: post.title,
      description: post.description,
      hashtags: post.hashtags,
      author: post.author,
      imageUrls: localImages,
    });
  }
}
```

What happens next?

- These imageuris actually contain the real bitmap binary data!
- Firebase Storage is not used here yet.

🏠 > posts > XXMyROG0Af1...

(default) ⓘ

+ Start collection

comments

posts >

📁 posts

+ Add document

4Ztek8HY0bvkAhjdS9J3

XXMyROG0Af1T9eQAJ6W >

nAjwzL00nXVgxKRuRRF0

📄 XXMyROG0Af1T9eQAJ6W

+ Start collection

+ Add field

author: "tester"

▼ comments

0 "3J42V8GgdLGSA0rkMxoG"

1 "aPrBnoJaGoVQt2dnXa8O"

Type

0 string ▼

String

data:image/png;base64,iVBORw0KGgoAAAANSU
hEUgAAoAAAAHgCAYAAAA1odzAAAQAEIEQVR
4AZT9ebct2XXdic254t77usx82QAgSNqSPoP/sOS
S2IEkCIAASRVVHhqSxQ49qBr2cElsVFWuUucqFS
UNiSBayh/M9YeGJFog2uzztffG9m+uHXHOufe9B
KQdMMB1e6/dB...4edE7Wd/0/M...0f5Ei-S...hxf

Display images

- Images will be displayed in both postDetails/[id].tsx and Post.tsx

```
/* — Image gallery — */
/* These URIs point at files on this device. */
{imageUrls.length > 0 && (
  <ScrollView
    horizontal
    showsHorizontalScrollIndicator={false}
    style={styles.imageScroll}
    contentContainerStyle={styles.imageScrollContent}
  >
    {imageUrls.map((url, i) => (
      <Image
        key={i}
        source={{ uri: url }}
        style={styles.detailImage}
        resizeMode="cover"
      />
    ))}
  </ScrollView>
)}
```

ASSIGNMENT 6

Add Bookmarks and My Favorite Tab

YOUR TASKS

- Add a bookmark button on the Post Detail screen and create a new My Favorite tab that shows only the posts bookmarked by the current user.
- Use `arrayUnion` / `arrayRemove` for toggling bookmarks.
- `AsyncStorage` for reading the current user.

DELIVERABLE

My Favorites tab only displays the posts bookmarked by that user.

EST. EFFORT

≈ 345 hours